```
import os
import random
import shutil
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from PIL import Image
from sklearn.metrics import classification_report

# Set seeds for reproducibility
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow version:", tf.__version__)
print("GPU available:", tf.config.list_physical_devices('GPU'))
```

```
TensorFlow version: 2.19.0
GPU available: []
```

```
BASE_DIR  = '/content/drive/MyDrive/AI L6/FruitinAmazon/FruitinAmazon'
TRAIN_DIR = os.path.join(BASE_DIR, 'train')
TEST_DIR  = os.path.join(BASE_DIR, 'test')

print("Train directory:", TRAIN_DIR)
print("Test  directory:", TEST_DIR)
print("Train classes  :", os.listdir(TRAIN_DIR))
```

```
Train directory: /content/drive/MyDrive/AI L6/FruitinAmazon/FruitinAmazon/train
Test  directory: /content/drive/MyDrive/AI L6/FruitinAmazon/FruitinAmazon/test
Train classes  : ['acai', 'cupuacu', 'graviola', 'pupunha', 'tucuma', 'guarana']
```

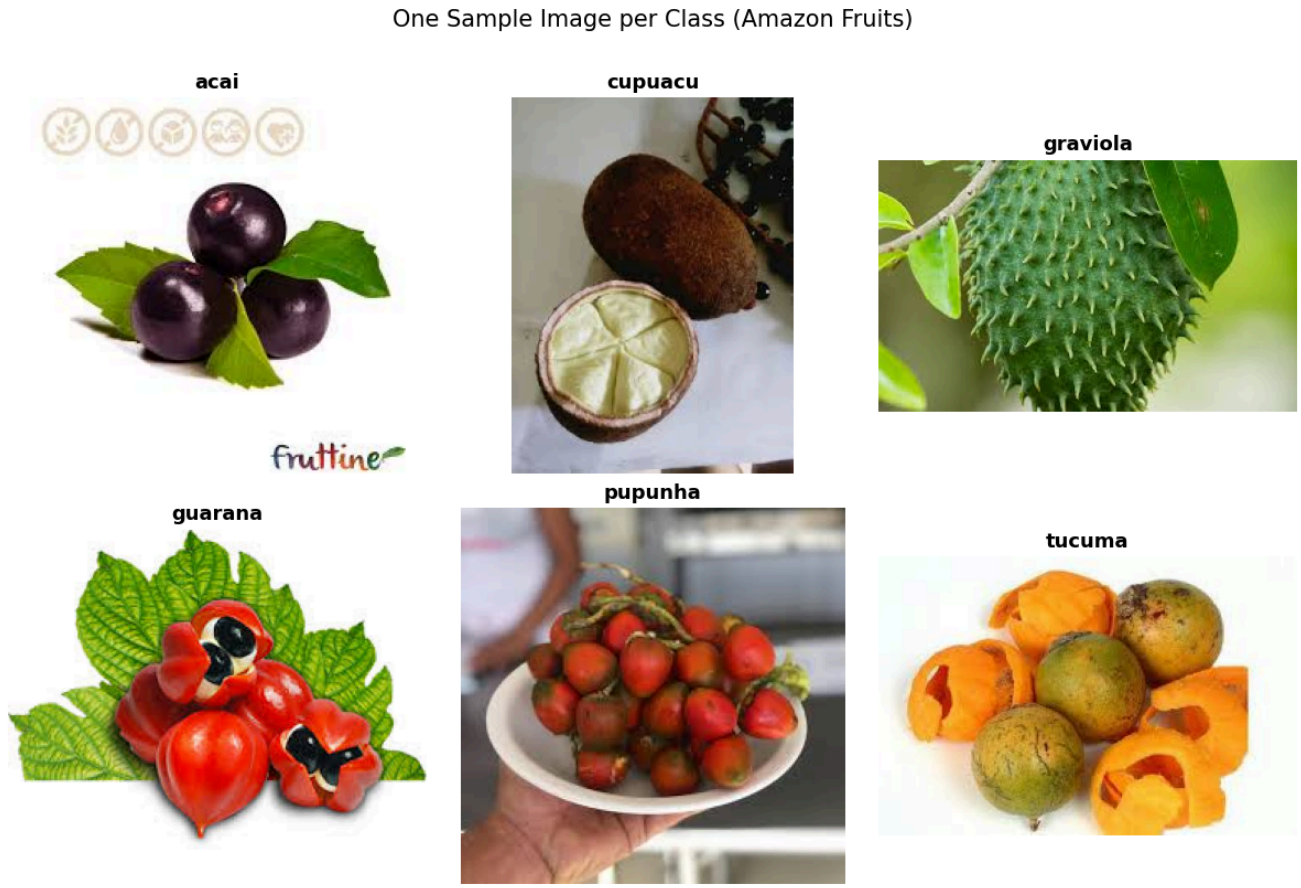
Task 1: Data Understanding and Visualization:

1. Load and visualize images from a dataset stored in directories, where each subdirec- tory represents a class.

```
# Get sorted list of class directories
class_dirs = sorted([d for d in os.listdir(TRAIN_DIR)
                     if os.path.isdir(os.path.join(TRAIN_DIR, d))])
num_classes = len(class_dirs)
print(f"Found {num_classes} classes: {class_dirs}")

# Select one random image per class
selected = []
for cls in class_dirs:
    cls_path = os.path.join(TRAIN_DIR, cls)
    images   = [f for f in os.listdir(cls_path)
                if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    chosen   = random.choice(images)
    selected.append((cls, os.path.join(cls_path, chosen)))

# Display in a grid with 2 rows
cols      = (num_classes + 1) // 2   # ceiling division for 2 rows
fig, axes = plt.subplots(2, cols, figsize=(cols * 4, 8))
axes      = axes.flatten()

for idx, (cls_name, img_path) in enumerate(selected):
    img = Image.open(img_path).convert('RGB')
    axes[idx].imshow(img)
    axes[idx].set_title(cls_name, fontsize=13, fontweight='bold')
    axes[idx].axis('off')

# Hide any unused subplot slots
for idx in range(len(selected), len(axes)):
    axes[idx].axis('off')

plt.suptitle('One Sample Image per Class (Amazon Fruits)', fontsize=15, y=1.01)
plt.tight_layout()
plt.savefig('class_samples.png', bbox_inches='tight', dpi=150)
plt.show()
```

```
print("\nObservation: The dataset contains images of Amazon fruits with varying backgrounds,"
      " lighting conditions, and image sizes. Each class shows distinct colour and texture"
      " patterns which a CNN can learn to distinguish.")
```

```
Found 6 classes: ['acai', 'cupuacu', 'graviola', 'guarana', 'pupunha', 'tucuma']
```




```
Observation: The dataset contains images of Amazon fruits with varying backgrounds, lighting conditions, and image sizes. Each
```

2. Check for Corrupted Image:

```
corrupted_images = []

for cls in class_dirs:
    cls_path = os.path.join(TRAIN_DIR, cls)
    for fname in os.listdir(cls_path):
        image_path = os.path.join(cls_path, fname)
        if not fname.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp')):
            continue
        try:
            with Image.open(image_path) as img:
                img.verify()          # checks for truncation / format errors
        except (IOError, SyntaxError):
            corrupted_images.append(image_path)
            os.remove(image_path)
            print(f"Removed corrupted image: {image_path}")

if not corrupted_images:
    print("No corrupted images found.")
else:
    print(f"\nTotal corrupted images removed: {len(corrupted_images)}")
```

```
No corrupted images found.
```

Task 2: Loading and Preprocessing Image Data in keras:

```
 #Hyper-parameters
IMG_HEIGHT       = 128
```

```
IMG_HEIGHT       = 128
IMG_WIDTH        = 128
BATCH_SIZE       = 16      # as specified in Task 4
VALIDATION_SPLIT = 0.2     # 80 / 20 split
SEED             = 123
```

```
# Normalisation layer (pixel values 0-255 → 0-1)
rescale = tf.keras.layers.Rescaling(1.0 / 255)

# ── Training dataset ──────────────────────────────────────────────────────
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    TRAIN_DIR,
    labels           = 'inferred',
    label_mode       = 'int',
    image_size       = (IMG_HEIGHT, IMG_WIDTH),
    interpolation    = 'nearest',
    batch_size       = BATCH_SIZE,
    shuffle          = True,
    validation_split = VALIDATION_SPLIT,
    subset           = 'training',
    seed             = SEED
)

# Class names (inferred from directory structure) should be retrieved here
class_names = train_ds.class_names
NUM_CLASSES  = len(class_names)
print("Class names :", class_names)
print("Num classes :", NUM_CLASSES)

train_ds = train_ds.map(lambda x, y: (rescale(x), y))

# ── Validation dataset
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    TRAIN_DIR,
    labels           = 'inferred',
    label_mode       = 'int',
    image_size       = (IMG_HEIGHT, IMG_WIDTH),
    interpolation    = 'nearest',
    batch_size       = BATCH_SIZE,
    shuffle          = False,
    validation_split = VALIDATION_SPLIT,
    subset           = 'validation',
    seed             = SEED
)
val_ds = val_ds.map(lambda x, y: (rescale(x), y))

# ── Test dataset
test_ds = tf.keras.preprocessing.image_dataset_from_directory(
    TEST_DIR,
    labels        = 'inferred',
    label_mode    = 'int',
    image_size    = (IMG_HEIGHT, IMG_WIDTH),
    interpolation = 'nearest',
    batch_size    = BATCH_SIZE,
    shuffle       = False
)
test_ds = test_ds.map(lambda x, y: (rescale(x), y))

# Performance optimisation
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().prefetch(buffer_size=AUTOTUNE)
val_ds   = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
test_ds  = test_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

```
Found 90 files belonging to 6 classes.
Using 72 files for training.
Class names : ['acai', 'cupuacu', 'graviola', 'guarana', 'pupunha', 'tucuma']
Num classes : 6
Found 90 files belonging to 6 classes.
Using 18 files for validation.
Found 30 files belonging to 6 classes.
```

Task 3 - Implement a CNN with Follow the following Structure and Hyper - parameters: Convolutional Architecture: • Convolutional Layer 1: – Filter Size (F): (3, 3) – Number of Filters (k): 32 – Padding (P): same – Stride (s): 1 • Activation Layer: ReLU activation • Pooling Layer 1:

Max pooling – Filter Size (F): (2, 2) – Stride (s): 2 • Convolutional Layer 2: – Filter Size (F): (3, 3) – Number of Filters (k): 32 – Padding (P): same – Stride (s): 1 • Activation Layer: ReLU activation • Pooling Layer 2: Max pooling – Filter Size (F): (2, 2) – Stride (s): 2

```
def build_cnn(num_classes: int, img_h: int, img_w: int) -> keras.Model:
    """Build the CNN model as described in the worksheet."""
    model = keras.Sequential([
        # ── Convolutional Block 1 ──────────────────────────────────────
        layers.Conv2D(
            filters     = 32,
            kernel_size = (3, 3),
            padding     = 'same',
            strides     = (1, 1),
            activation  = 'relu',
            input_shape = (img_h, img_w, 3),
            name        = 'conv1'
        ),
        layers.MaxPooling2D(
            pool_size = (2, 2),
            strides   = (2, 2),
            name      = 'pool1'
        ),

        # ── Convolutional Block 2 ──────────────────────────────────────
        layers.Conv2D(
            filters     = 32,
            kernel_size = (3, 3),
            padding     = 'same',
            strides     = (1, 1),
            activation  = 'relu',
            name        = 'conv2'
        ),
        layers.MaxPooling2D(
            pool_size = (2, 2),
            strides   = (2, 2),
            name      = 'pool2'
        ),

        # ── Fully Connected Network ────────────────────────────────────
        layers.Flatten(name='flatten'),

        layers.Dense(64,  activation='relu', name='dense_hidden1'),
        layers.Dense(128, activation='relu', name='dense_hidden2'),

        layers.Dense(num_classes, activation='softmax', name='output')
    ], name='CNN_FruitClassifier')

    return model


model = build_cnn(NUM_CLASSES, IMG_HEIGHT, IMG_WIDTH)
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_sh
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "CNN_FruitClassifier"

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 128, 128, 32)	896
pool1 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2 (Conv2D)	(None, 64, 64, 32)	9,248
pool2 (MaxPooling2D)	(None, 32, 32, 32)	0
flatten (Flatten)	(None, 32768)	0
dense_hidden1 (Dense)	(None, 64)	2,097,216
dense_hidden2 (Dense)	(None, 128)	8,320
output (Dense)	(None, 6)	774

Total params: 2,116,454 (8.07 MB)
Trainable params: 2,116,454 (8.07 MB)
Non-trainable params: 0 (0.00 B)

Task 4: Compile the Model

```
model.compile(
    optimizer = 'adam',
    loss      = 'sparse_categorical_crossentropy',
    metrics   = ['accuracy']
)
print("Model compiled successfully.")
```

```
Model compiled successfully.
```

```
# Callbacks
checkpoint_cb = keras.callbacks.ModelCheckpoint(
    filepath          = 'best_model.h5',
    monitor           = 'val_accuracy',
    save_best_only    = True,
    save_weights_only = False,
    mode              = 'max',
    verbose           = 1
)

early_stop_cb = keras.callbacks.EarlyStopping(
    monitor              = 'val_loss',
    patience             = 20,        # stop if no improvement for 20 epochs
    restore_best_weights = True,
    verbose              = 1
)

reduce_lr_cb = keras.callbacks.ReduceLROnPlateau(
    monitor  = 'val_loss',
    factor   = 0.5,
    patience = 10,
    min_lr   = 1e-6,
    verbose  = 1
)

print("Callbacks configured: ModelCheckpoint, EarlyStopping, ReduceLROnPlateau")
```

```
Callbacks configured: ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
```

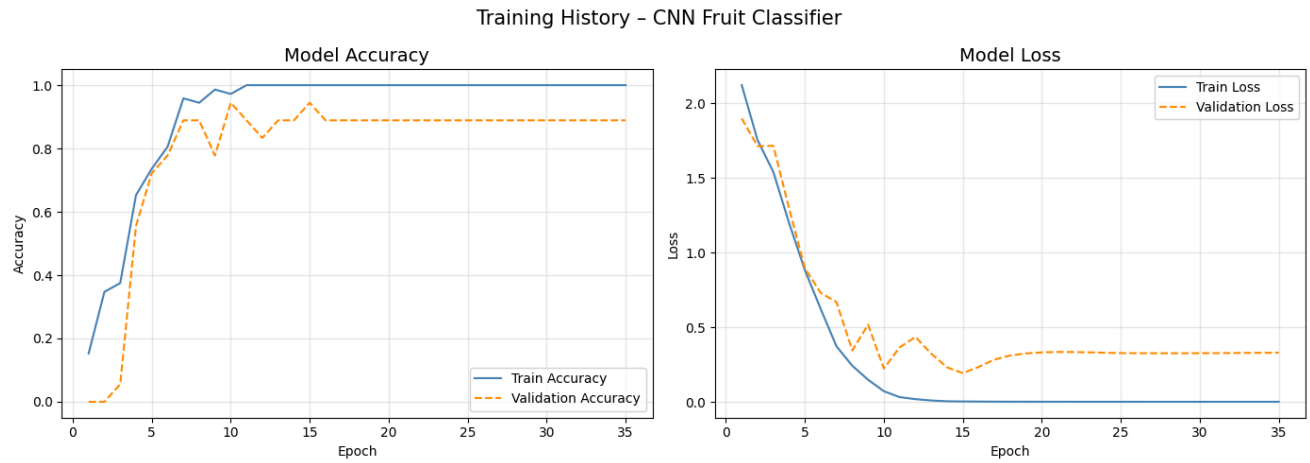
Task 4: Train the Model

```
# Train
EPOCHS = 250

history = model.fit(
    train_ds,
    epochs          = EPOCHS,
    validation_data = val_ds,
    callbacks       = [checkpoint_cb, early_stop_cb, reduce_lr_cb],
    verbose         = 1
)

print(f"\nTraining finished at epoch {len(history.history['loss'])}.")
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 274ms/step - accuracy: 1.0000 - loss: 4.0350e-04
Epoch 28: val_accuracy did not improve from 0.94444
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 301ms/step - accuracy: 1.0000 - loss: 3.8722e-04 - val_accuracy: 0.8889 - val_loss: 0.3253 - lear
Epoch 29/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 268ms/step - accuracy: 1.0000 - loss: 3.9243e-04
Epoch 29: val_accuracy did not improve from 0.94444
5/5 ━━━━━━━━━━━━━━━━━━━━ 1s 294ms/step - accuracy: 1.0000 - loss: 3.7642e-04 - val_accuracy: 0.8889 - val_loss: 0.3254 - lear
Epoch 30/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 263ms/step - accuracy: 1.0000 - loss: 3.8191e-04
Epoch 30: val_accuracy did not improve from 0.94444
5/5 ━━━━━━━━━━━━━━━━━━━━ 1s 292ms/step - accuracy: 1.0000 - loss: 3.6613e-04 - val_accuracy: 0.8889 - val_loss: 0.3258 - lear
Epoch 31/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 266ms/step - accuracy: 1.0000 - loss: 3.7189e-04
Epoch 31: val_accuracy did not improve from 0.94444
5/5 ━━━━━━━━━━━━━━━━━━━━ 1s 290ms/step - accuracy: 1.0000 - loss: 3.5634e-04 - val_accuracy: 0.8889 - val_loss: 0.3262 - lear
Epoch 32/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 270ms/step - accuracy: 1.0000 - loss: 3.6230e-04
Epoch 32: val_accuracy did not improve from 0.94444
5/5 ━━━━━━━━━━━━━━━━━━━━ 1s 294ms/step - accuracy: 1.0000 - loss: 3.4696e-04 - val_accuracy: 0.8889 - val_loss: 0.3270 - lear
Epoch 33/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 471ms/step - accuracy: 1.0000 - loss: 3.5302e-04
Epoch 33: val_accuracy did not improve from 0.94444
5/5 ━━━━━━━━━━━━━━━━━━━━ 4s 552ms/step - accuracy: 1.0000 - loss: 3.3792e-04 - val_accuracy: 0.8889 - val_loss: 0.3278 - lear
Epoch 34/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 355ms/step - accuracy: 1.0000 - loss: 3.4406e-04
Epoch 34: val_accuracy did not improve from 0.94444
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 382ms/step - accuracy: 1.0000 - loss: 3.2923e-04 - val_accuracy: 0.8889 - val_loss: 0.3285 - lear
Epoch 35/250
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 277ms/step - accuracy: 1.0000 - loss: 3.3537e-04
Epoch 35: val_accuracy did not improve from 0.94444

Epoch 35: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
5/5 ━━━━━━━━━━━━━━━━━━━━ 2s 303ms/step - accuracy: 1.0000 - loss: 3.2083e-04 - val_accuracy: 0.8889 - val_loss: 0.3293 - lear
Epoch 35: early stopping
Restoring model weights from the end of the best epoch: 15.

Training finished at epoch 35.
```

Training / Validation Loss & Accuracy Curves

```
def plot_history(history):
    epochs_ran = range(1, len(history.history['loss']) + 1)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # ── Accuracy ──
    ax1.plot(epochs_ran, history.history['accuracy'],     label='Train Accuracy',      color='steelblue')
    ax1.plot(epochs_ran, history.history['val_accuracy'], label='Validation Accuracy', color='darkorange', linestyle='--')
    ax1.set_title('Model Accuracy', fontsize=14)
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Accuracy')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # ── Loss ──
    ax2.plot(epochs_ran, history.history['loss'],     label='Train Loss',      color='steelblue')
    ax2.plot(epochs_ran, history.history['val_loss'], label='Validation Loss', color='darkorange', linestyle='--')
    ax2.set_title('Model Loss', fontsize=14)
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Loss')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.suptitle('Training History – CNN Fruit Classifier', fontsize=15)
    plt.tight_layout()
    plt.savefig('training_history.png', dpi=150, bbox_inches='tight')
    plt.show()

plot_history(history)
```



Task 5: Evaluate the Model

```
test_loss, test_acc = model.evaluate(test_ds, verbose=1)
print(f"\nTest Loss    : {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}  ({test_acc*100:.2f}%)")
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 86ms/step - accuracy: 0.8000 - loss: 0.6586

Test Loss    : 0.6586
Test Accuracy: 0.8000  (80.00%)
```

Task 6: Save and Load the Model

```
MODEL_PATH = '/content/drive/MyDrive/AI L6/FruitinAmazon/fruit_cnn_model.h5'
model.save(MODEL_PATH)
print(f"Model saved to '{MODEL_PATH}'")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file forma
Model saved to '/content/drive/MyDrive/AI L6/FruitinAmazon/fruit_cnn_model.h5'
```

```
#load
loaded_model = keras.models.load_model(MODEL_PATH)
print("Model loaded successfully.")

reloaded_loss, reloaded_acc = loaded_model.evaluate(test_ds, verbose=1)
print(f"\nReloaded Model – Test Loss    : {reloaded_loss:.4f}")
print(f"Reloaded Model – Test Accuracy: {reloaded_acc:.4f}  ({reloaded_acc*100:.2f}%)")
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty u
Model loaded successfully.
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 83ms/step - accuracy: 0.8000 - loss: 0.6586

Reloaded Model – Test Loss    : 0.6586
Reloaded Model – Test Accuracy: 0.8000  (80.00%)
```

Task 7: Predictions and Classification Report

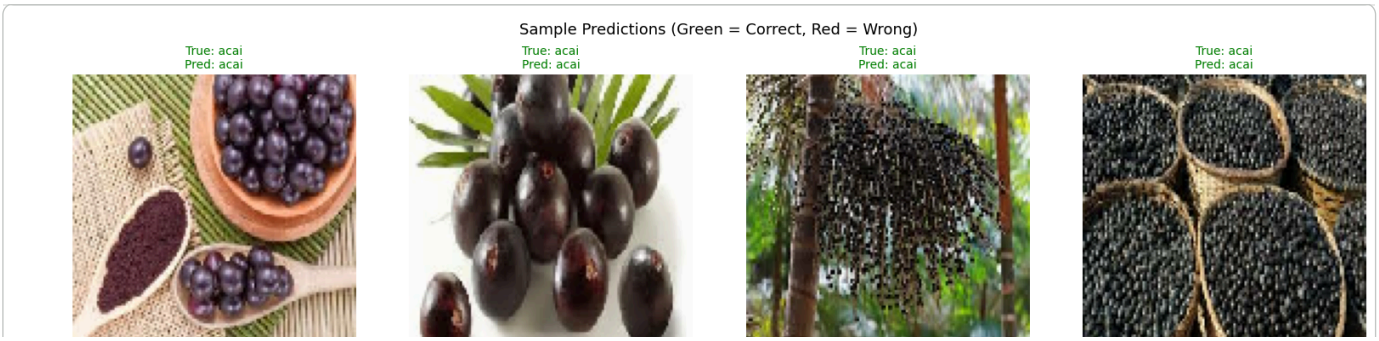
```
#  Collect all true labels and predictions
y_true  = []
y_pred  = []

for images, labels in test_ds:
    preds  = loaded_model.predict(images, verbose=0)
    y_pred.extend(np.argmax(preds, axis=1))
    y_true.extend(labels.numpy())

y_true = np.array(y_true)
y_pred = np.array(y_pred)

print(f"Total test samples: {len(y_true)}")
print(f"Overall Accuracy  : {np.mean(y_true == y_pred):.4f}")
```

```
Total test samples: 30
Overall Accuracy  : 0.8000
```

```
#  Classification Report
report = classification_report(y_true, y_pred, target_names=class_names)
print("Classification Report")
print("=" * 60)
print(report)
```

```
Classification Report
============================================================
              precision    recall  f1-score   support

        acai       0.71      1.00      0.83         5
     cupuacu       1.00      0.80      0.89         5
    graviola       0.83      1.00      0.91         5
     guarana       0.67      0.80      0.73         5
     pupunha       0.75      0.60      0.67         5
      tucuma       1.00      0.60      0.75         5

    accuracy                           0.80        30
   macro avg       0.83      0.80      0.80        30
weighted avg       0.83      0.80      0.80        30
```

```
#Visualise a few sample predictions
sample_images, sample_labels = next(iter(test_ds))
sample_preds   = loaded_model.predict(sample_images, verbose=0)
sample_pred_cls = np.argmax(sample_preds, axis=1)

n_show = min(8, len(sample_images))
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.flatten()

for i in range(n_show):
    axes[i].imshow(sample_images[i].numpy())
    true_cls = class_names[sample_labels[i]]
    pred_cls = class_names[sample_pred_cls[i]]
    colour   = 'green' if true_cls == pred_cls else 'red'
    axes[i].set_title(f"True: {true_cls}\nPred: {pred_cls}",
                      color=colour, fontsize=10)
    axes[i].axis('off')

plt.suptitle('Sample Predictions (Green = Correct, Red = Wrong)', fontsize=13)
plt.tight_layout()
plt.savefig('sample_predictions.png', dpi=150, bbox_inches='tight')
plt.show()
```


Sample Predictions (Green = Correct, Red = Wrong)
True: acai
Pred: acai
True: acai
Pred: acai
True: acai
Pred: acai
True: acai
Pred: acai